Writing a Test

This test has a few parts. Let's take a second to plan the given/when/then:

- Given: You'll need one existing project and at least one existing task on that project so that you can test the ordering. You'll probably want two tasks; that way you can verify that the user interface is correct for the first, last, and middle parts of the list.
- When/then: The user fills out the form for the task and you verify that the new task shows up.
- When/then: The user moves a task up and you verify the order changes.

The fact that you have two distinct when/then pairs suggests this is probably two tests, but for ease of explanation let's keep it as one and see how it looks:

integration/01/spec/system/add_task_spec.rb

```
require "rails_helper"

RSpec.describe "adding a new task" do
  let!(:project) { create(:project, name: "Project Bluebook") }
  let!(:task_1) { create(
    :task, project: project, title: "Search Sky", size: 1) }
  let!(:task_2) { create(
    :task, project: project, title: "Use Telescope", size: 1) }

  it "can add and reorder a task" do
    visit(project_path(project))
    fill_in("Task", with: "Find UFOs")
    select("2", from: "Size")
    click_on("Add Task")
    expect(current_path).to eq(project_path(project))
    within("#task_3") do
      expect(page).to have_selector(".name", text: "Find UFOs")
      expect(page).to have_selector(".size", text: "2")
      expect(page).not_to have_selector("a", text: "Down")
      click_on("Up")
    end
    expect(current_path).to eq(project_path(project))
    within("#task_2") do
      expect(page).to have_selector(".name", text: "Find UFOs")
    end
  end

end
```

The test is kind of long and rambling. It also doesn't completely test the UI, in the sense that it's not validating that all the "Up" and "Down" links that are supposed to be there are actually there—you might do that in a helper or